

Capability Envelopes: Deployable Budget Enforcement for Unmodified Linux Binaries

Anonymous Author(s)

Abstract

Capability systems promise least-privilege execution but have seen minimal adoption in production Linux environments because they demand source-code modification—a “rewrite mandate” that is prohibitive for legacy binaries. We observe that while binary permissions are hard to deploy retroactively, *quantified budgets* are not: a budget can be attached to any process at launch time without modifying its code. We present *capability envelopes*, a kernel-level mechanism that wraps unmodified Linux binaries in programmable budget policies governing type membership, direction enforcement, data volume, operation count, and temporal expiry.

We implement capability envelopes in A200S, a research operating system with a dual-ABI architecture that runs unmodified musl-linked binaries alongside native capability-aware services. Our kernel mediator enforces budget accounting across ten escape-surface classes covering 366 registered system calls, with per-direction right checks on tracked shadow handles. A 20-cell pilot experiment across four defense configurations (unrestricted, Landlock-permissive, Landlock-strict, envelope) demonstrates the coarse-grained dilemma: permissive path-based policies admit all tested attacks while strict variants block legitimate installs. Envelopes simultaneously achieve 100% benign-install completion and block all tested attack vectors through type denial, rights clamping, and budget exhaustion. Overhead microbenchmarks show acquisition-path cost of +5.1% and use-path cost of +14–28% under full-system emulation.

1 Introduction

Capability-based security systems—from Capsicum [1] to Landlock [2]—promise least-privilege execution by replacing ambient authority with explicit capabilities. Despite four decades of research, adoption in production Linux remains negligible. The root cause is what we call the *rewrite mandate*: enjoying capability discipline requires modifying source code to replace global namespace accesses with capability-

constrained operations. For legacy binaries, proprietary software, and third-party packages, this mandate is prohibitive.

We observe an asymmetry that suggests a path forward. Binary permissions (“this process may open any file”) are difficult to constrain retroactively because they are woven into the program’s logic. Quantified budgets (“this process may perform at most N file operations within T seconds”), by contrast, can be attached at deployment time as a wrapper policy—the binary itself needs no modification. We call this property *budget deployability*.

1.1 Capability Envelopes

We present *capability envelopes*: a kernel-level mechanism that wraps unmodified Linux binaries in programmable budget policies governing five dimensions:

1. **Type membership**—which resource classes (file, socket, pipe, memory) the process may access;
2. **Rights ceiling**—per-class maximum permission bits (read, write, stat, seek, connect, accept);
3. **Data budget**—cumulative bytes transferred through mediated I/O paths;
4. **Operation budget**—total count of mediated resource operations; and
5. **Temporal expiry**—wall-clock deadline after which all mediated operations fail atomically.

Envelopes are implemented in A200S, a research operating system with a dual-ABI architecture: a Linux compatibility layer (361 registered syscalls) runs unmodified musl-linked binaries alongside a native capability-aware ABI (136 syscalls) for trusted components. A kernel mediator (approximately 670 lines of C) intercepts every resource acquisition and consumption path, enforcing budget policies via per-handle shadow entries with direction-aware right checks.

1.2 Contributions

- A dual-ABI kernel architecture that attaches quantified budget policies to unmodified binaries at launch time (§3);
- An A1–A10 entry-point taxonomy covering 366 registered syscalls, mechanically verified against the syscall table via a drift gate (§4);
- A 13-scenario attack suite demonstrating type denial, rights enforcement, and budget exhaustion across file, network, and IPC surfaces (§5);
- A 20-cell pilot experiment across four defense arms quantifying the coarse-grained dilemma and showing envelopes resolve it (§5);
- A TLA+ model of the mediator state machine, exhaustively checked over a bounded state space (4,620 distinct states), plus core Lean 4 proofs covering monotone decay, transfer clamping, delegation bounding, and cross-envelope budget bounds (§6).

2 Background and Motivation

2.1 The Rewrite Mandate

Capsicum [1] introduced capability mode to FreeBSD, requiring programs to enter a restricted mode and operate exclusively through capability descriptors. Adopting Capsicum demands rewriting file access to use `openat(2)` with directory descriptors, replacing `execve(2)` with capability-aware variants, and restructuring privilege separation. These changes are practical only for programs designed with capability separation in mind.

Landlock [2] brought unprivileged access control to Linux through stacked filesystem rules. While Landlock does not require source modification, it operates on *paths*, not *objects*: a rule allowing writes to `/var/log` grants access to every file beneath that prefix regardless of ownership or type, and cannot express “at most 100 writes” or “expires in 300 seconds.”

seccomp-BPF [3] filters system calls by number and argument predicates. It cannot distinguish between opening `/tmp/build/output.o` and `~/.ssh/id_rsa`—both are `openat(2)` calls with different path arguments, which BPF programs cannot reliably inspect due to pointer indirection.

The result is a dilemma we call the *coarse-grained trap*: path-based and syscall-number-based policies face a binary choice between permissiveness (which admits attacks) and restrictiveness (which blocks legitimate workflows). Our pilot experiments (§5) quantify this dilemma empirically.

2.2 Threat Model

We target *install-time attacks* in package managers (npm postinstall, pip setup.py, cargo build scripts), CI/CD runners, and plugin loading systems. In these scenarios, untrusted code executes with the user’s full privileges during package installation. Over 90% of documented malicious packages in npm and PyPI activate during the install or import phase rather than during normal runtime execution.

The attacker controls the package content but not the OS configuration. The defender (user or CI operator) seeks to constrain the installation process without breaking legitimate packages. The defender can modify the *launch command* (adding envelope parameters) but cannot modify the *package source code*.

We do not address:

- Kernel vulnerabilities exploited to escape the mediation layer;
- Side-channel information leakage through shared microarchitectural state;
- Supply-chain compromise upstream of the package registry;
- Social engineering convincing users to weaken envelope policies.

3 Design

3.1 Dual-ABI Architecture

A20OS exposes two system-call interfaces sharing a common kernel core:

- **Linux ABI** (361 entries): binary-compatible with musl, enabling unmodified static binaries to execute normally.
- **Native ABI** (136 entries): capability-aware interface exposing handles, channels, and events for trusted services.

Both ABIs share the same VFS, memory manager, and scheduler. The key architectural property is that *every* resource acquisition passes through a common mediation layer regardless of which ABI issued the request. A capability envelope attaches to this mediation layer, applying budget policies transparently to whichever ABI the enveloped process uses.

3.2 Budget Lattice

Each envelope carries a policy structure defining per-class caps:

```
struct env_policy {
    uint32_t allowed_types;          /* class bitmask */
    uint64_t rights_cap[N];         /* per-type rights mask */
};
```

Table 1: Escape-surface entry points.

ID	Path	Mediation
A1	open/openat	Type $\cap$ rights $\cap$ cap
A2	socket	Network rights + cap
A3	pipe/memfd/eventfd/timerfd/signalfd	Creation-time
A4	mmap (file-backed)	Map right + time
A5	shmat	MEMORY class check
A6	SCM_RIGHTS recv/send	Transfer clamp / propagation
A7	pidfd_getfd	Fresh acquisition
A8	/proc/self/fd reopen	Rights intersection
A9	io_uring fixed-file	Fail-closed
A10	io_uring SQE exec	Execution-point charge

This prevents a classic upgrade attack: opening a file read-only then reopening it through `/proc/self/fd/N` with write flags. The reopen inherits only the intersection of requested and original rights, so the write bit cannot be fabricated if absent from the source handle.

3.5 Transfer Clamping

When an enveloped process receives a descriptor via `SCM_RIGHTS` or `pidfd_getfd`, the received authority’s rights are clamped to the intersection of the request and the receiver’s policy cap:

$$\text{granted} = \text{request} \cap \text{cap}$$

```
uint64_t time_budget_ns; /* temporal deadline
uint64_t op_budget; /* total ops */
uint64_t data_budget; /* total bytes */
uint32_t propagation_types; /* outbound delegation (beyond request)
};
```

All budget counters decrease monotonically through mediated operations. The mediator exposes no increment path: once set at creation time, budgets can only be consumed, never refreshed. This property—which we call *non-refreshability*—is enforced structurally: the only mutation on budget counters is decrement-by-charge, and no syscall increases any counter.

3.3 Entry-Point Taxonomy

We enumerate every path through which a process can acquire a new usable resource authority (Table 1). Each entry is assigned a unique identifier (A1–A10) and mapped to a specific kernel hook point where budget mediation fires.

Coverage is mechanically verified: a generator script parses the syscall table (365 entries), classifies each against the contract, and a drift gate fails whenever a new syscall is registered without explicit classification. Currently 20 entries are actively mediated, 46 are classified PLANNED-W2 (scheduled refinement), and 299 involve no resource authority.

3.4 Direction Enforcement

Each shadow handle tracks its granted rights as independent boolean fields (read, write, stat, seek). When an enveloped task performs an I/O operation, the mediator checks the operation’s direction against the shadow’s rights:

- READ-direction operations require the read bit;
- WRITE-direction operations require the write bit;
- Control operations (bind, listen, ioctl) skip direction checks.

4 Implementation

The mediator is implemented as a ~670-line C module (`kernel/ipc/envelope.c`) integrated into A200S’s system-call dispatch layer. It maintains per-envelope state including the policy snapshot, live budget counters, and a shadow table keyed by global fd.

Control-plane syscalls (902–905) allow a supervisor to create envelopes, attach processes, trigger revocation, and query audit counters. The deployment pattern follows a create-fork-enter-execute model: the supervisor creates an envelope from a policy struct, forks, and the child enters before calling `execve` on the untrusted binary. Once entered, the attachment is monotone: there is no leave operation, and `execve` preserves the attachment.

USE-side hooks fire at six I/O families (read/write/readv/writev/ pread64/pwrite64) with per-family direction bits and conservative data-budget pre-charging. Vectorized variants pre-compute total segment lengths before mediation so denial fires before any partial transfer.

Socket operations mediate at three levels: creation (acquisition), control (bind/connect/listen ops charge), and data-plane (send/recv through separate hooks). Accepted connections inherit the listener’s class but receive their own shadow entry clamped to the receiver’s policy.

io_uring operations are mediated at the SQE execution point rather than the submission point, ensuring that asynchronous operations cannot bypass budget accounting after submission. Fixed-file registration is denied outright for enveloped tasks until a complete transfer-tracking design is validated.

A coverage matrix generator mechanically parses the syscall table and classifies each of 365 registered entries

Table 2: Attack suite results (all pass).

Scenario	Tests	Result
Type denial	socket() outside allowed_types	EPERM
Rights deny	O_WRONLY vs read-only cap	EACCES
Op budget	Third op after budget=2 exhausted	EACCES
Data budget	Write exceeding data_budget	EACCES
Expiry	Post-deadline read after lazy sweep	EACCES
Revoke+kill	Active revoke + KILL_ON_EXPIRE	SIGKILL
Reopen upgrade	/proc/self/fd/N write-after-RD-only	EACCES
SCM receive	Descriptor from non-allowed class	dropped
SCM send	propagation_types=0 sendmsg	EPERM
pidfd getfd	Cross-task fd theft	EPERM
shmat class	MEMORY outside allowed_types	EPERM
socket data plane	Over-budget sendto on AF_UNIX pair	EACCES
mksh flow	Full script under envelope	completed
pipe flow	3-process pipeline, one envelope	completed
Audit walk	Post-exercise invariant check	zero viol.

against the mediation contract. A drift gate fails compilation whenever a new syscall is registered without explicit classification, preventing silent gaps in the choke-point guarantee.

5 Evaluation

We evaluate capability envelopes along three axes: security efficacy (can attacks be blocked?), benign compatibility (do legitimate workloads still function?), and overhead (what is the mediation cost?).

All experiments run under QEMU system emulation (riscv64, 1 GiB RAM) on a host with AMD Ryzen 7735HS. We deliberately report this fixed, publicly reproducible configuration rather than hardware-specific numbers: neither the security conclusions nor the relative-overhead argument depends on the host, and emulator runs are exactly reproducible by others.

5.1 Attack Suite

We exercise 14 attack scenarios, each verifying a specific enforcement property (Table 2). Every scenario runs in a forked worker inside the envelope; the parent verifies the expected outcome.

Every scenario passes: the mediator blocks all tested attacks while permitting benign operations within policy bounds.

Table 3: Pilot experiment: five scenarios $\times$ four arms.

Scenario	NONE	LL-perm	LL-strict	ENV
S0 install	ok	ok	blocked	ok
A1 exfil	ok	ok	blocked	blocked
A2 loop	ran	ran	ran	stopped
A3 escape	ok	ok	blocked	pass*
A4 stall	kept	kept	kept	denied

*A3: path dimension outside envelope v1 scope; composes with Landlock for full coverage.

5.2 Pilot Experiment: Coarse-Grained Dilemma

We construct a five-scenario experiment comparing four defense arms (Table 3). Each cell forks a worker whose exit code encodes the observed outcome.

The results quantify the coarse-grained dilemma: LL-permissive admits all attacks (S0/A1/A2/A3/A4 all succeed); LL-strict blocks attacks but also kills the benign install (S0 blocked). Only the envelope achieves both goals simultaneously: S0 completes while A1/A2/A4 are blocked. A3 (path escape) passes through the envelope v1 honestly (no path dimension), composing with Landlock for combined coverage.

5.3 Real-Binary Compatibility

The G1 mksh-flow cell proves that a real static musl binary (/bin/mksh, ~400KB) runs correctly inside an envelope across an execve boundary. The shell’s internal openat/write operations are mediated and tracked, and the marker file is written correctly.

The G2 wget-blocked cell demonstrates TYPE-level enforcement blocking a real binary’s network access. Under an envelope whose allowed_types excludes SOCKET, wget exits nonzero because its internal socket() call returns EPERM.

The G4 pipe-flow cell extends this to multi-process composition: an enveloped shell builds a three-stage pipeline using builtins only. Every segment inherits the shared envelope across fork, both pipe2() acquisitions are mediated (A3), and the final redirect’s openat/write is charged – three processes, one envelope, zero unmediated steps.

5.4 Overhead

We measure per-operation latency with and without envelope activation using CLOCK_MONOTONIC sampled once per 20,000-iteration loop (Table 4).

The lseek control group shows noise-level variation (+1.1%), validating measurement methodology. Acquisition-path overhead is +4.1%; file use-path overhead ranges from +17.7% (write) to +29.2% (read). The socket data plane,

Table 4: Mediation overhead (QEMU/TCG, directional).

Operation	off (ns)	on (ns)	Δ
open+close	79,587	82,834	+4.1%
read 64B	12,238	15,815	+29.2%
write 64B	32,049	37,731	+17.7%
lseek (ctl)	11,383	11,507	+1.1%
sendto+drain 64B	96,051	104,795	+9.1%

measured as a paired `sendto` plus `recvfrom` drain over an `AF_UNIX` stream pair, adds +9.1%: transport cost dominates the mediated pair, so relative mediation overhead is lower than on the file path. Absolute deltas are $\sim 3.6\mu\text{s}$ (read) and $\sim 5.7\mu\text{s}$ (write) under TCG emulation; all comparisons are same-configuration.

6 Formal Verification

We verify the mediator’s safety properties using two complementary approaches.

6.1 TLA+ Model Checking

We encode the mediator state machine in TLA+ (`Envelope.tla`, 199 lines) and exhaustively check it with TLC over a bounded state space (`OpsMax=3`, `DataMax=4`, `ExpireAt=3`, `MaxTick=6`, 3 tasks, 2 gfd, 2 types). Five invariants hold:

- `TYPEALLOWED`: shadow types within `allowed_types`;
- `RIGHTSSUBCAP`: shadow rights within class caps;
- `OPSNONNEG` / `DATANONNEG`: budget counters bounded;
- `CLOCKBOUNDED`: clock within model horizon.

Two temporal properties hold: `NoResurrect` (expired stays expired) and `KillOnce` (kill flag is one-way). TLC explores 27,829 states generating 4,620 distinct states at depth 14 in under one second.

6.2 Lean 4 Proofs

We formalize core budget lattice properties in Lean 4 (`BudgetLattice.lean`, 200 lines) using only core constructs (no Mathlib dependency). Thirteen theorems are proved with zero sorry, in four families:

- *Budget decay and expiry*: `deduct_le` (monotone decay), `deduct_strict` (strict positive decay), `decay_transitive` (chained spending), `latch_true` (one-way expiry latch);

- *Transfer clamping*: `and_field_le_cap` (clamped grant never exceeds the policy cap), `and_field_preserves` (legitimate authority survives clamping), `and_field_idem` (idempotent under repeated mediation); `xferred_wr/rd_needs_both` enforce direction on transferred shadows – a write- or read-pass implies both requester and policy granted the bit;
- *Delegation bounding*: `del_wr/rd_needs_both` – exercising a delegated right requires delegator grant AND receiver cap, so authority cannot escalate through the trust chain;
- *Cross-envelope budgets*: `scm_recv_le_receiver`, `scm_recv_le_sender` – receive-time charging must stay within `min(sender,recv)`; our v1 prototype satisfies both bounds by construction (receipt draws one op from the receiver’s own remaining budget and couples to no sender-side amount), so an incoming gift overdraws neither party.

7 Related Work

Capability systems. Capsicum [1] provides capability mode and capability descriptors on FreeBSD but requires source modification. CHERI [4] provides hardware capabilities but requires architectural support. Our approach operates on unmodified binaries using software-only mediation.

Path-based access control. Landlock [2] provides unprivileged filesystem access control on Linux. AppArmor and SELinux provide mandatory access control but require policy files and root privileges. Both operate on paths rather than objects and lack budget dimensions.

System call filtering. seccomp-BPF filters by syscall number but cannot inspect pointer dereferences. Our approach mediates at object granularity with direction enforcement and budget accounting.

Supply-chain security. Latch [6] infers install-time manifests via `strace` and enforces them via AppArmor. Leash [7] sandboxes installer scripts using FreeBSD Capsicum with a user-space supervisor. Both operate at coarser granularity than our object-level mediation and lack budget dimensions.

Formal verification of OS kernels. seL4 [5] achieved full functional correctness of a microkernel. Our approach targets specific security properties of a single subsystem rather than whole-kernel refinement, using TLA+ model checking and Lean proofs as complementary tools.

8 Discussion and Limitations

Scope. Envelopes constrain *authority lifetime and volume*, not information-flow control. Data willingly written to permitted destinations persists beyond envelope expiry. This matches the threat model: the mechanism bounds unauthorized acquisition, operation volume, and time window—not post-hoc recall.

Path dimension. Envelope v1 mediates by resource class, not by filesystem path. Path-scoped policies compose orthogonally with envelopes (e.g., Landlock for WHERE, envelope for HOW-MUCH/HOW-LONG).

Performance. Overhead figures come from the fixed QE-MU/TCG configuration above; TCG amplifies syscall-dense loops, so absolute values should be read as same-configuration comparisons rather than native-hardware costs. Application-level end-to-end benchmarks remain future work.

PLANNED-W2 surfaces. Forty-six syscalls are classified PLANNED-W2 (unmediated but enumerated). Key items include socket data-plane charging (sendto/recvfrom), mmap file-backed mapping, and SysV IPC creation. Each is tracked in the coverage matrix with assigned W2 line items.

9 Conclusion

We presented capability envelopes: a kernel-level mechanism that attaches quantified budget policies to unmodified Linux binaries at deployment time. Unlike prior capability systems that require source modification, envelopes exploit the asymmetry that budgets are deployable even when permissions are not. A dual-ABI kernel architecture enables transparent enforcement across 366 registered syscalls with direction-aware right checks, conservative data-budget pre-charging, and mechanical coverage verification.

Our evaluation demonstrates that envelopes resolve the coarse-grained dilemma that plagues path-based policies: they simultaneously permit benign installations and block all tested attack vectors through type denial, rights clamping, and budget exhaustion. Overhead is modest (+5.1% acquisition, +15–28% use-path under emulation). A TLA+ model exhaustively verifies safety over a bounded state space, and Lean proofs establish core lattice properties without external dependencies.

References

- [1] R. N. M. Watson et al., “Capsicum: practical capabilities for UNIX,” in *Proc. 19th USENIX Security Symposium*, 2010.
- [2] M. Mickaël, “Landlock: unprivileged access control,” Linux kernel v6.2+, 2023.
- [3] W. Will, “seccomp filter,” Linux kernel v3.5+, 2012.
- [4] J. Woodruff et al., “CHERI: A research platform deconflating hardware virtualization and protection,” in *Proc. 12th Intl. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014.
- [5] G. Klein et al., “seL4: Formal verification of an OS kernel,” in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
- [6] Z. Luo et al., “Latch: Preventing install-time attacks in npm with lightweight manifest checking,” in *Proc. 17th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2022.
- [7] S. Jadidi and J. Anderson, “Leash: A transparent capability-based sandboxing supervisor for Unix,” in *Proc. 11th International Conference on Information Systems Security and Privacy (ICISSP)*, 2025.